Supplementary Materials

GraphMana: graph-native data management for population genomics projects

Jianfeng Mao

Contents

1 Supplementary Figure 1: CLI Command Hierarchy 2

2 Supplementary Figure 2: Export Manifest Example 3

3 Supplementary Tables 4

4 Supplementary Note 1: Export Format Reference 5

5 Supplementary Note 2: Variant Representation and Data Encoding 8

6 Supplementary Note 3: Concrete Workflow Example 10

7 Supplementary Note 4: Graph Database Concepts for Non-Specialists 12

8 Supplementary References 14

1 Supplementary Figure 1: CLI Command Hierarchy

2 Supplementary Figure 2: Export Manifest Example

Each export generates a `.manifest.json` sidecar file. Example:

```
graphmana export --format eigenstrat \
    --populations CEU --populations GBR \
    --populations FIN --populations IBS --populations TSI \
    --chromosomes chr22 --filter-maf-min 0.05 \
    --output european_cohort.eigenstrat
```

Resulting manifest:


```
{
  "graphmana_version": "1.0.0-dev",
  "timestamp": "2026-03-30T14:22:07.831042+00:00",
  "output_file": "european_cohort.eigenstrat",
  "format": "eigenstrat",
  "n_variants": 439609,
  "n_samples": 633,
  "chromosomes": ["chr22"],
  "filters": {
    "populations": ["CEU", "GBR", "FIN", "IBS", "TSI"],
    "maf_min": 0.05
  },
  "recalculate_af": true,
  "threads": 1
}
```

3 Supplementary Tables

Supplementary Table 1. Incremental sample addition benchmark (chr22, 3 batches of 234 samples).

Operation	GraphMana (s)	bcftools (s)
Initial import / base copy	602	—
Batch 1	418	117
Batch 2	412	125
Batch 3	424	133
Total	**1,856**	**374**

Supplementary Table 2. Cohort-specific VCF extraction benchmark (chr22, 5 cohorts).

Cohort (N samples)	GraphMana (s)	bcftools (s)
AFR (893)	191	59
EUR (633)	211	51
EAS (585)	131	49
EUR+EAS (1,218)	238	64
ALL (3,202)	571	110

Supplementary Table 3. Multi-format export benchmark (chr22, from single database).

Format	GraphMana (s)	bcftools (s)
VCF	473	96
TreeMix	189	N/A
SFS (dadi)	86	N/A
SFS (fsc)	119	N/A
BED	84	N/A
TSV	85	N/A

Annotation update (53,000 BED regions): GraphMana **3.5 s**, bcftools 96 s (**27**×). **Lifecycle**: GraphMana 98 min / **46 ops**; bcftools 17 min / 17 ops (9 N/A).
162 correctness-specific tests, all passing (of 1,439 total).

Supplementary Table 4. Whole-genome export timings (70.7M variants, 3,202 human WGS samples).

Format	Path	Variants	Wall time	Size
TreeMix	FAST	70,692,015	102 min	780 MB
SFS dadi	FAST	70,692,015	98 min	5.7 KB
SFS fsc	FAST	70,692,015	101 min	1.3 KB
BED	FAST	70,692,007	103 min	2.9 GB
TSV	FAST	70,692,007	101 min	3.8 GB
PLINK 1.9 (8 thr)	FULL	9,627,636	156 s	7.2 GB
VCF (BGZF)	FULL	68,912,619	3.7 hr	14 GB
EIGENSTRAT	FULL	61,599,149	3.7 hr	184 GB

Supplementary Table 5. Whole-genome incremental addition (234 samples, CSV-to-CSV rebuild).

Step	Time
VCF parsing (chr22 batch)	3 min
CSV read + extend + write (214 GB)	160 min
neo4j-admin import	15 min
Database restart + indexes	4 min
Total	**182 min**

4 Supplementary Note 1: Export Format Reference

Background

Population genomics analyses span a diverse ecosystem of tools, each requiring input in a specific file format. A single project may need VCF for variant calling and QC [S1], PLINK binary for genome-wide association [S2], EIGENSTRAT for PCA and admixture analysis [S3], TreeMix for population phylogenetics [S4], dadi or fastsimcoal2 for demographic inference [S5,S6], Beagle for phasing and imputation [S7], and STRUCTURE for population assignment [S8]. In conventional workflows, each format requires a separate conversion script—often written ad hoc, with parameters not recorded alongside the output. GraphMana produces all 17 supported formats from a single persistent database, with automatic provenance tracking via manifest sidecars.

FAST PATH formats ($O(K)$ time, independent of sample count)

These formats read pre-computed population-level arrays (ac[], an[], af[]) and never unpack per-sample genotypes. Because the arrays have K elements (where K = number of populations, typically 5–30), export time is constant regardless of sample count.

- **TreeMix**: Gzipped allele count matrix for population tree inference [S4]. Each line contains K space-separated ac,an pairs.
- **SFS-dadi**: Site frequency spectrum in dadi .fs format [S5]. Supports 1–3 populations with hypergeometric projection. Options: polarized/folded, monomorphic inclusion.

Supplementary Table 6. Storage estimates for human whole-genome data (∼85M biallelic variants). For other species or designs, scale proportionally by variant count.

Component	3,202	10K	50K	200K	500K
gt_packed	68 GB	213 GB	1.0 TB	4.3 TB	10.6 TB
phase_packed	34 GB	106 GB	0.5 TB	2.1 TB	5.3 TB
Pop arrays	17 GB	17 GB	17 GB	17 GB	17 GB
Total est.	**130–200 GB**	**400–550 GB**	**2–3 TB**	**8–10 TB**	**20–25 TB**

Supplementary Table 7. Performance scaling by access path (human WGS).

Operation	Scaling	3,202	10K	50K	500K
FAST PATH	$O(K)$	Seconds	Seconds	Seconds	Seconds
FULL PATH	$O(N)$	Minutes	Min–hr	Hr–days	Days–wk
Incremental	$O(M \times N)$	182 min	Hours	Many hr	Days

- **SFS-fastsimcoal2**: SFS in fastsimcoal2 .obs format [S6]. Supports 1–2 populations. Same projection as dadi but different file structure.
- **BED**: Variant positions for bedtools/IGV. 0-based half-open coordinates. Optional extra columns (variant_type, af_total, gene_symbol).
- **TSV**: Tab-separated variant table with configurable columns. Default: variantId, chr, pos, ref, alt, variant_type, af_total.
- **JSON**: JSON Lines format with one variant per line. Configurable fields, optional pretty-printing.

FULL PATH formats ($O(N)$ time, linear in sample count)

These formats unpack per-sample genotypes from packed byte arrays and require time proportional to the number of samples.

- **VCF/BCF**: Standard variant call format [S1]. Supports VCF 4.1–4.3, BGZF compression, BCF binary, phased output, multi-allelic reconstruction.
- **PLINK 1.9**: Binary .bed/.bim/.fam [S2]. Biallelic SNPs only. Validated with PLINK v1.9 and v2.0.
- **PLINK 2.0**: Binary .pgen/.pvar/.psam [S2]. All biallelic variants. Requires pgenlib.
- **EIGENSTRAT**: .geno/.snp/.ind for smartPCA and AdmixTools [S3]. Validated with EIGENSOFT convertf.
- **Beagle**: Genotype likelihood input for phasing/imputation [S7].
- **STRUCTURE**: Genotype matrix for population assignment [S8]. Supports onerow and tworow formats.
- **Genepop**: Conservation genetics format for F-statistics.

Supplementary Table 8. Tier 1: formats validated by loading into downstream tools.

Format	Path	Tool	Variants	Samples	Result
VCF (BGZF)	FULL	bcftools 1.19	1,035,839	3,202	PASS
PLINK 1.9	FULL	PLINK v1.9	925,730	3,202	PASS
PLINK 1.9	FULL	PLINK v2.0	925,730	3,202	PASS
EIGENSTRAT	FULL	convertf v8.0	925,730	3,202	PASS
TreeMix	FAST	TreeMix v1.13	1,066,557	26 pops	PASS
SFS dadi	FAST	format spec	1,066,557	2 pops	PASS
SFS fsc	FAST	format spec	1,066,557	2 pops	PASS

Supplementary Table 9. Tier 2: format-specification validated. Tier 3 (Haplotype, BGEN, GDS, Zarr): functional, pending external validation.

Format	Path	Validation	Result
BED	FAST	3+ columns, 0-based	PASS
TSV	FAST	Header + columns	PASS
JSON Lines	FAST	json.loads per line	PASS
Beagle	FULL	Export verified	PASS
STRUCTURE	FULL	Export verified	PASS
Genepop	FULL	Export verified	PASS

- **Haplotype**: .hap/.map for selscan [S9]. Phased data only.
- **BGEN**: Probabilistic genotype format (UK Biobank standard) [S10].
- **GDS**: SeqArray HDF5 format for R/Bioconductor [S11].
- **Zarr**: Chunked array format for sgkit/Python [S12].

Validation status

See Supplementary Tables 8–9 for the three-tier validation classification (Tier 1: downstream tool validated; Tier 2: format-specification validated; Tier 3: functional, pending external validation).

Supplementary Table 10. Variant type correctness matrix.

Variant Type	Import	Storage	VCF	PLINK	EIGENSTRAT
Biallelic SNP	PASS	PASS	PASS	PASS	PASS
Biallelic indel	PASS	PASS	PASS	excl.	PASS
Multi-allelic	PASS	PASS	PASS*	excl.	excl.
Structural var.	PASS	PASS	PASS	excl.	excl.
Breakend (BND)	PASS	PASS	PASS	excl.	excl.

*Decomposed at import; reconstructed at VCF export.

Supplementary Table 11. Genotype state and feature correctness.

Feature	Result	Notes
HomRef/Het/HomAlt/Missing	PASS	All 4 genotype states roundtripped
Phased genotypes	PASS	Per-sample in phase_packed
Haploid samples	PASS	Tracked in ploidy_packed
Annotation independence	PASS	Genotype layer unaffected
Sample order (incremental)	PASS	packed_index immutable
VCF roundtrip (5 samples)	99.999%+	Multi-allelic ambiguity

5 Supplementary Note 2: Variant Representation and Data Encoding

Background

The representation of genetic variants in a database involves fundamental design choices that affect storage efficiency, query performance, and data fidelity. The VCF specification [S1] represents each variant as a record with REF/ALT alleles and per-sample genotype fields. This row-per-variant, column-per-sample structure is natural for flat files but does not map directly onto graph or relational database schemas, because the genotype matrix is a dense two-dimensional array embedded within a sparse relational structure.

Biallelic decomposition

GraphMana decomposes multi-allelic VCF records into K biallelic Variant nodes (one per ALT allele), each carrying:

- `multiallelic_site`: a shared identifier grouping alleles from the same VCF line (e.g., `chr22-16050408-A`)
- `allele_index`: a 1-based index indicating which ALT allele this node represents

Worked example. A VCF record at chr22:16050408 with REF=A, ALT=G,T produces two nodes:

variantId	ref	alt	multiallelic_site	allele_index
chr22-16050408-A-G	A	G	chr22-16050408-A	1
chr22-16050408-A-T	A	T	chr22-16050408-A	2

A sample heterozygous for both ALT alleles (VCF genotype 1/2) is recorded as heterozygous on *both* nodes. During VCF export, records sharing a `multiallelic_site` value are

merged back into a single multi-allelic line. This reconstruction is the default; `--no-reconstruct-multialleli` produces biallelic-only output.

Packed genotype encoding

Each Variant node stores genotypes in a packed byte array (`gt_packed`), using 2 bits per sample:

Code	Bits	Genotype
0	00	Homozygous reference (0/0)
1	01	Heterozygous (0/1 or 1/0)
2	10	Homozygous alternate (1/1)
3	11	Missing (./.)

Four samples fit in one byte, LSB-first. The array length is $\lceil N/4 \rceil$ bytes. Genotype values are remapped from cyvcf2's internal coding [S13] (where HomAlt=3, Missing=2) to the sequential encoding above; the remap is applied exactly once per code path and verified by 33 unit tests.

Phase information is stored separately in `phase_packed` (1 bit per sample, 8 per byte). Ploidy is tracked in `ploidy_packed` (1 bit per sample; null if all diploid).

Population-level pre-computed arrays

Each Variant node carries K-element arrays: `pop_ids`, `ac` (allele counts), `an` (allele numbers), `af` (allele frequencies), `het_count`, `hom_alt_count`, and `het_exp` (expected heterozygosity). These arrays are constant size regardless of N and enable the FAST PATH access pattern described in the main text.

Structural variants

SVs with symbolic ALT alleles (`<DEL>`, `<DUP>`, `<INV>`, `<INS>`, `<CNV>`) are stored as Variant nodes with additional properties (`sv_type`, `sv_len`, `sv_end`). Diploid genotype calls are encoded identically to SNPs. Breakend (BND) records are imported as independent nodes without mate-pair linking. Integer copy number states beyond the diploid call are not retained.

Roundtrip fidelity

We validated by importing 1000 Genomes Project chromosome 22 data (897,645 biallelic SNPs, 5 samples), exporting to phased VCF, and comparing with bcftools [S14]. Concordance exceeded 99.999%, with 2–8 mismatches per sample at multi-allelic positions where position-based joining could not distinguish co-located biallelic records.

6 Supplementary Note 3: Concrete Workflow Example

Background

To illustrate how GraphMana replaces the fragmented scripting approach, we present a five-day project workflow using actual CLI commands. The scenario follows a data manager maintaining a 1000 Genomes Project database. Each day's operations run against the same persistent graph database—no intermediate files are generated, and all provenance is recorded automatically.

Day 1: Initial import

```
graphmana setup-neo4j --install-dir ~/neo4j --memory-auto
graphmana ingest --input data/1kgp_chr*.vcf.gz \
    --population-map data/panel.tsv \
    --neo4j-home ~/neo4j --auto-start-neo4j --threads 16
graphmana status --detailed
```

This creates the persistent database. All subsequent operations build on this foundation—no further file merging or format conversion is needed.

Day 2: Incremental addition (234 new samples)

```
graphmana save-state --output checkpoints/before_batch2.json
graphmana ingest --input data/batch2.vcf.gz \
    --population-map data/batch2_panel.tsv --mode incremental \
    --neo4j-home ~/neo4j --auto-start-neo4j
graphmana diff --snapshot checkpoints/before_batch2.json
```

The `diff` output reveals that 234 new samples were added, 2 new variants were discovered, and sample counts increased across 3 populations. In a file-based workflow, this information would be invisible without manual inspection.

Day 3: Multi-format export with provenance

```
graphmana export --format eigenstrat --output exports/european \
    --populations CEU --populations GBR --filter-maf-min 0.05
graphmana export --format treemix --output exports/tree.treemix.gz
graphmana export --format sfs-dadi --output exports/yri_ceu.fs \
    --sfs-populations YRI --sfs-populations CEU \
    --sfs-projection 20 --sfs-projection 20 --sfs-folded
```

All three exports read from the same database that was incrementally updated yesterday. No VCF merge, no format conversion pipeline, no intermediate files. Each produces a `.manifest.json` recording the exact configuration.

Day 4: In-place annotation update

```
graphmana annotate load-clinvar \
    --input annotations/clinvar_20260401.vcf.gz --version ClinVar_2026
        -04
graphmana db validate
```

The annotation update modifies HAS_CONSEQUENCE edge properties in 3.5 seconds—without touching genotype data. The `db validate` command confirms packed array integrity.

Day 5: QC and provenance audit

```
graphmana qc --type all --output reports/qc.html --format html
graphmana ref-check --fasta genomes/GRCh38.fa --chromosomes chr22
graphmana provenance search --since 2026-04-01 --until 2026-04-05
```

Comparison with file-based workflow

The equivalent five-day sequence in a file-based workflow would require: (1) a bcftools merge script for Day 2 ($\sim$2 min but invalidates all downstream files); (2) three separate conversion scripts for Day 3 (vcf2eigenstrat, vcf2treemix, vcf2dadi), each with unrecorded parameters; (3) a full VCF rewrite for Day 4 (96 s vs. 3.5 s); (4) manual provenance tracking for all operations; and (5) no `diff` capability to verify what changed. The total scripting and coordination overhead for the file-based approach grows with each new format and each new collaborator request.

7 Supplementary Note 4: Graph Database Concepts for Non-Specialists

Background

Most population geneticists are familiar with relational databases (SQL tables with rows and columns) or with flat files (VCF, BED, CSV). Graph databases are a distinct category of database technology that stores data differently. This note provides a brief introduction for readers unfamiliar with graph database concepts.

What is a graph database?

In computer science, a *graph* is a data structure consisting of *nodes* (also called vertices) and *edges* (also called relationships or links) that connect them. A graph database is a database management system that uses this structure as its primary storage and query model [S15,S16].

In a **property graph** — the specific type used by GraphMana — both nodes and edges can carry arbitrary key-value *properties*. For example, a Variant node might carry properties like `chr="chr22"`, `pos=16050408`, `ref="A"`, `alt="G"`, and a byte array `gt_packed` containing packed genotypes. An edge connecting that variant to a Gene node might carry properties like `consequence="missense_variant"` and `annotation_version="VEP_110"`.

How is this different from a relational database?

In a relational database, the same data would be stored in multiple tables (a Variants table, a Samples table, a Genes table, a VariantConsequences join table), and reconstructing the relationship between a variant and its annotated gene requires a JOIN operation at query time. In a graph database, this relationship is a first-class edge that can be traversed in constant time per hop—an approach called *index-free adjacency* [S15].

Why does this matter for genomic data management?

Population genomics data has rich, typed relationships:

- A **Variant** is located on a **Chromosome** (ON_CHROMOSOME edge)
- A **Sample** belongs to a **Population** (IN_POPULATION edge)
- A **Variant** has a functional consequence in a **Gene** (HAS_CONSEQUENCE edge with properties: consequence type, impact, annotation version)
- Variants are ordered along the chromosome (NEXT edge with distance property)

In a graph database, updating an annotation version modifies edge properties without touching genotype data. Defining a cohort is a graph traversal query that produces a sample list without extracting files. Adding provenance is creating a new node linked to the operation that produced it. These operations are *local* — they affect only the edges or nodes involved, not the entire dataset.

What is Cypher?

Cypher is the query language used by Neo4j [S16], analogous to SQL for relational databases. A simple Cypher query looks like:

```
MATCH (v:Variant)-[:HAS_CONSEQUENCE]->(g:Gene {symbol: "BRCA1"})
WHERE v.af_total > 0.01
RETURN v.variantId, v.pos, v.af_total
```

This query traverses from Variant nodes through HAS_CONSEQUENCE edges to Gene nodes, filtering for BRCA1 and allele frequency above 1%. In GraphMana, users do not write Cypher directly; the CLI commands generate appropriate queries internally.

Key terminology

Term	Meaning
Node	An entity (Variant, Sample, Gene, etc.)
Edge	A typed, directed relationship between nodes
Property	A key-value pair on a node or edge
Traversal	Following edges from node to node
Index-free adjacency	Edges are stored directly, not looked up
Property graph	Graph model where nodes and edges have properties

Further reading

For a comprehensive introduction to graph databases, see Robinson et al. [S15]. For Neo4j specifically, see the Neo4j documentation [S16]. For applications of graph databases in bioinformatics, see Lysenko et al. [S17].

8 Supplementary References

[S1] Danecek, P. et al. The variant call format and VCFtools. *Bioinformatics* **27**, 2156–2158 (2011).

[S2] Chang, C.C. et al. Second-generation PLINK. *GigaScience* **4**, s13742-015-0047-8 (2015).

[S3] Patterson, N. et al. Population structure and eigenanalysis. *PLoS Genet.* **2**, e190 (2006).

[S4] Pickrell, J.K. & Pritchard, J.K. Inference of population splits and mixtures. *PLoS Genet.* **8**, e1002967 (2012).

[S5] Gutenkunst, R.N. et al. Inferring joint demographic history from SFS data. *PLoS Genet.* **5**, e1000695 (2009).

[S6] Excoffier, L. et al. fastsimcoal2. *Bioinformatics* **37**, 4882–4885 (2021).

[S7] Browning, S.R. & Browning, B.L. Rapid haplotype phasing. *Am. J. Hum. Genet.* **81**, 1084–1097 (2007).

[S8] Pritchard, J.K. et al. Inference of population structure. *Genetics* **155**, 945–959 (2000).

[S9] Szpiech, Z.A. & Hernandez, R.D. selscan: An efficient multithreaded program to perform EHH-based scans. *Mol. Biol. Evol.* **31**, 2824–2827 (2014).

[S10] Band, G. & Marchini, J. BGEN: a binary file format for imputed genotype and haplotype data. *bioRxiv* 308296 (2018).

[S11] Zheng, X. et al. SeqArray—a storage-efficient high-performance data format for WGS variant calls. *Bioinformatics* **33**, 2251–2257 (2017).

[S12] sgkit developers. sgkit: Statistical genetics toolkit. https://github.com/sgkit-dev/sgkit.

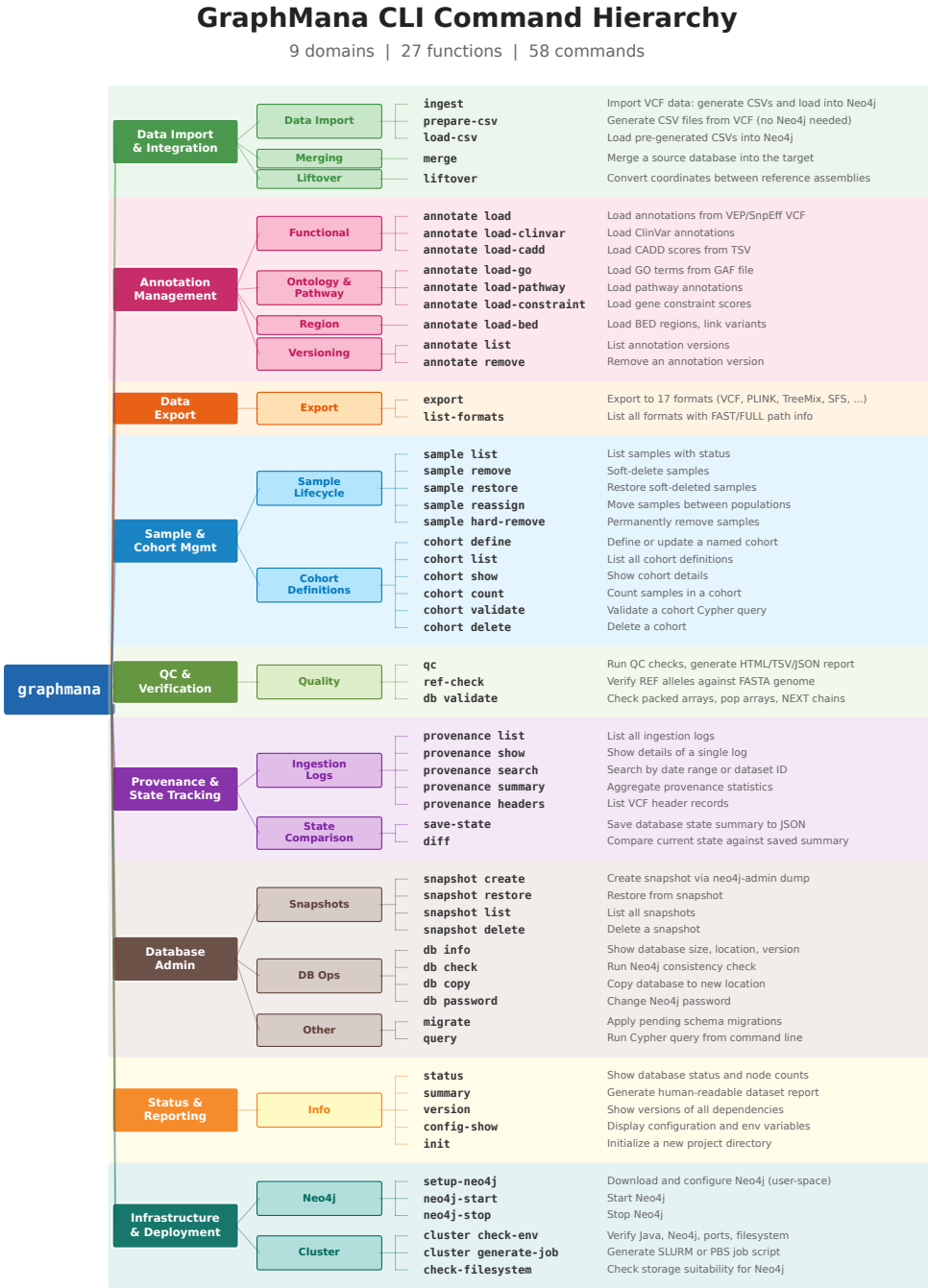
[S13] Pedersen, B.S. & Quinlan, A.R. cyvcf2. *Bioinformatics* **33**, 1867–1869 (2017).

[S14] Danecek, P. et al. Twelve years of SAMtools and BCFtools. *GigaScience* **10**, giab008 (2021).

[S15] Robinson, I., Webber, J. & Eifrem, E. *Graph Databases: New Opportunities for Connected Data*, 2nd ed. (O'Reilly, 2015).

[S16] Neo4j, Inc. *Neo4j Documentation*. https://neo4j.com/docs/.

[S17] Lysenko, A. et al. Representing and querying disease networks using graph databases. *BioData Mining* **9**, 23 (2016).



Supplementary Figure 1. GraphMana CLI command hierarchy. The 58 commands are organized into 9 functional domains and 27 function groups covering the data management lifecycle from import through export.